Data Challenge Kaggle - Kernel methods

Alexandre Mallez & Rayane Dakhlaoui (MVA/IASD 2025-2026) Github project :
https://github.com/AlexHibo/Kaggle-challenge-kernel-methods

Dataset

The dataset consists of 5 000 training and 2 000 test images of size 32×32 pixels in colour (CIFAR-10, 10 balanced classes of 500 images each). A first visual inspection immediately revealed a noisy label problem: even within the training set, a non-negligible fraction of images appeared visually misclassified (e.g. a dog labelled as a cat), putting an empirical ceiling on achievable accuracy. This motivated us to prefer soft-margin classifiers and feature-averaging strategies robust to label noise.

Approach Progression

Step 1 — Raw-pixel Kernel SVM (val. acc. ≈ 25%)
Our baseline was a soft-margin SVM (One-vs-All, dual QP via `cvxopt`) trained directly on the raw 3 072-dimensional pixel vectors. We experimented with several kernels (linear, polynomial, chi-squared, histogram intersection, and RBF) and found that the **RBF kernel** consistently outperformed all others, a pattern that held throughout all subsequent experiments. Despite this, accuracy stalled around 25%: pixel space carries almost no invariance to translation, lighting, or viewpoint.

Step 2 — HOG + RBF-SVM, with augmentation (val. acc. ≈ 57–59%)
We replaced raw pixels with **Histogram of Oriented Gradients (HOG)** descriptors (9 orientation bins, 4×4-pixel cells, 2×2-cell blocks with L2-Hys normalisation), producing 1 764-dimensional vectors per image. The HOG pipeline extracts multi-channel gradients, keeps the channel with maximum magnitude per pixel, builds soft-binned orientation histograms per cell, and normalises overlapping blocks for illumination invariance.

Coupled with an RBF-SVM, this yielded a large jump to ≈56% accuracy at a regularization parameter C=20. We then added **data augmentation**: each training image is extended with its horizontal flip and a random crop (reflection padding of 4 pixels, then a random 32×32 crop), and a random subsample of the augmented pool is drawn for training. This pushed accuracy to **≈59%**, at the cost of a significantly longer training time due to the larger kernel matrix.

Step 3 — HOG + Kernel Ridge Regression (val. acc. ≈ 62%)
We experimented with **Kernel Ridge Regression (KRR)** as an alternative to the SVM. KRR solves the multi-class problem in one shot by minimising $\|\hat{Y} - Y\|_F^2 + \lambda \operatorname{tr}(\alpha^\top K \alpha)$, with closed-form solution $\alpha = (K + \lambda I)^{-1} Y_{\text{one-hot}}$ exploited via a Cholesky solve. Again, RBF dominated other kernel choices. On HOG features with augmentation (λ=0.05), KRR reached ≈62%, slightly above the SVM, while being faster (no QP). This became our default regressor for subsequent steps.

Step 4 — CKN features + KRR (val. acc. ≈ 63%)
We replaced HOG with **CKN features** (see Section 3 for full details). With 256 k-means filters, SPM levels $(1, 2, 4)$, PCA to 1 024 components, and KRR (λ=0.05), CKN alone achieved ≈63%, marginally above HOG+KRR. We also attempted to *train* the CKN filters via backpropagation in NumPy, but this proved prohibitively slow and plateaued at ≈50% after 150 epochs; it yielded an issue in our implementation. In addition to the long time of execution, we abandoned this path.

Step 5 — CKN + HOG fusion + PCA + KRR (val. acc. ≈ 69%, Kaggle ≈ 70%)
Since HOG captures fine texture and edge patterns while CKN encodes more semantic, spatially pooled features, we merged both representations by concatenating their standardised feature vectors with a mixing weight α: $\phi = [\alpha \cdot \phi_{\text{CKN}},\ (1-\alpha) \cdot \phi_{\text{HOG}}]$. A grid search over α with 512 k-means CKN filters and augmentation gave the best validation accuracy at α=0.3 (**≈69%**), confirming that HOG complements CKN . After PCA reduction to 2 048 components and KRR (λ=0.05), this full pipeline was used for the final submission, achieving **≈70% on the Kaggle public leaderboard**.

Final Pipeline — CKN + HOG + PCA + KRR

The central theoretical motivation for the CKN is that it provides a finite-dimensional feature map approximating a convolutional kernel. For two L2-normalised patches x, x' on the unit sphere, the arc-cosine kernel of order 1 is $k(x, x') = \frac{1}{\pi}(\sin\theta + (\pi-\theta)\cos\theta)$ where $\theta = \arccos(x^\top x')$. This is efficiently approximated by the randomised map $\varphi(x) = \mathrm{ReLU}(Wx)$ with L2-normalised filters W: $\mathbb{E}[\varphi(x)^\top \varphi(x')] \approx k(x, x')$. Using k-means centroids as filters concentrates the approximation budget on the directions of highest data density, giving a better approximation than random Gaussian filters.

The full CKN thus implicitly defines a kernel over images by computing local patch-level arc-cosine similarities, aggregating them spatially via pooling and SPM, and composing scales, all expressible as inner products in a reproducing kernel Hilbert space. The outer RBF kernel on CKN features then operates in this rich, hierarchically-structured space, which explains why the combination consistently outperforms shallower representations.

CKN feature extraction

For each image, the following operations are applied (implemented from scratch in NumPy):

- **Patch extraction.** Overlapping 3×3 patches on a dense grid yield 30×30=900 patches of dimension d=27 per image.
- **ZCA whitening.** $W_{\mathrm{ZCA}} = U\Lambda^{-1/2}U^\top$ estimated from subsampled training patches, applied identically to all views and test data. Decorrelating patch dimensions makes the dot-product a better proxy for the arc-cosine kernel.
- **L2 normalisation.** Projects each whitened patch onto the unit sphere, so that $\varphi(x)^\top\varphi(x')$ approximates a normalised arc-cosine kernel, removing magnitude dependency.
- **K-means filter bank & ReLU.** 512 filters from k-means++ (30 iterations) on whitened normalised patches, then L2-renormalised. The map $x \mapsto \mathrm{ReLU}(Wx)$ is the kernel feature approximation; each coordinate measures rectified similarity to one learned prototype.
- **Average pooling** (stride s=2) aggregates local responses, reducing the spatial grid from 30×30 to 15×15 and providing translation invariance.
- **Spatial Pyramid Matching.** Levels (1×1, 2×2, 4×4) average-pool the feature map in increasingly fine grids; descriptors are concatenated, giving dimension 512×21 = 10 752.
- **Power-norm + L2.** $\phi \leftarrow \mathrm{sign}(\phi)\sqrt{|\phi|}$, then ℓ_2-normalise. Compresses heavy tails and improves linear separability of the kernel feature map.

Feature fusion, PCA, and classification

CKN and HOG features are independently standardised and concatenated as $\phi = [0.3{\cdot}\phi_{\mathrm{CKN}},\ 0.7{\cdot}\phi_{\mathrm{HOG}}]$. PCA is applied via SVD of the centred training matrix; a diagnostic of cumulative explained variance (90% in ≈300 components, effective rank ≈192 out of 5 376) guides reduction to d=2 048 components, which acts both as compression and regularisation by discarding noisy directions. Finally, an RBF-kernel KRR (λ=0.05, $\gamma = 1/(d \cdot \mathrm{Var}(X))$ estimated on training data) is trained via a single Cholesky solve and used to predict 10-class labels on the test set.

Summary

Step	Method	Val. acc.	Key gain
1	Raw pixels + RBF-SVM	≈25%	baseline
2	HOG + RBF-SVM	≈57%	better features
2b	HOG + augmentation + RBF-SVM	≈59%	augmentation
3	HOG + augmentation + RBF-KRR	≈62%	faster regressor
4	CKN (256 filters) + PCA + RBF-KRR	≈63%	richer features
5	**CKN+HOG + PCA + RBF-KRR**	**≈69%**	**fusion**

Throughout all experiments, RBF systematically outperformed linear, polynomial, chi-squared and histogram-intersection kernels. The progression reflects a principled kernel-methods design: from pixel-space dot products, through hand-crafted invariant features (HOG), to an implicit convolutional RKHS (CKN), culminating in a fusion that reaches **≈69% on Kaggle**, all without backpropagation.